

Docker for IoT Systems

The Problem Without Docker

You built a working IoT backend on your laptop. Now you want to deploy it to a server.

Your laptop:

Python 3.11

Flask 3.0

PostgreSQL 16

paho-mqtt 2.0

Server:

Python 3.8

Flask 1.1

PostgreSQL 12

paho-mqtt 1.6

Result: ✗ "It works on my machine!"

More problems as the system grows:

Problem 1: "Works on my machine"

- Different OS, Python versions, library versions

Problem 2: Dependency conflicts

- Flask app needs Library A v1.0
- MQTT subscriber needs Library A v2.0
- Cannot install both on same machine!

Problem 3: Hard to replicate

- New team member spends days setting up environment

Problem 4: Hard to scale

- Need 3 copies of the API server? Manual setup 3 times.

What is Docker?

Docker packages your application and all its dependencies into a **container** — a self-contained, portable unit.

Without Docker:

App
+ Python 3.11
+ Flask 3.0
+ Libraries

Runs on: ???

→

With Docker:

Container
App
+ Python 3.11
+ Flask 3.0
+ Libraries

Runs identically everywhere.

A container is like a **shipping container** — same box, same contents, any ship

Container vs Virtual Machine

Virtual Machine:



Size: ~10 GB, boots in minutes

Container:



Size: ~100 MB, starts in seconds

Containers share the host OS kernel — much lighter than VMs.

Key Docker Concepts

Dockerfile	→ Recipe: "how to build this image"
Image	→ Built package: frozen snapshot of app + dependencies
Container	→ Running instance of an image
Registry	→ Storage for images (Docker Hub, GitHub Container Registry)

Workflow:

```
Write Dockerfile → docker build → Image
                                   ↓
                                docker run ▼
                                   Container (running app)
```

Think of it as:

- Dockerfile: Source code for the container
- Image: Compiled binary
- Container: Running processor
- Registry: App store for images

Dockerfile — Flask REST API

Dockerfile instructs how to build the image for our Flask API:

```
# Dockerfile for Flask REST API
FROM python:3.11-slim          # start from official Python image

WORKDIR /app                  # set working directory inside container

COPY requirements.txt .        # copy dependency list first (for caching)
RUN pip install -r requirements.txt # install dependencies

COPY . .                      # copy all application code

EXPOSE 3000                   # document that port 3000 is used

CMD ["gunicorn", "-w", "4", "-b", "0.0.0.0:3000", "main:app"]
```


Notice that the requirements.txt is copied and installed before the application code.

- This allows Docker to cache the layer with dependencies — so if you change your app code but not dependencies, it won't reinstall everything.

```
requirements.txt:
```

```
flask==3.0.0  
gunicorn==21.2.0  
psycpg2-binary==2.9.9  
paho-mqtt==2.0.0
```

Build and run:

```
docker build -t iot-api .          # build image tagged "iot-api"  
docker run -p 3000:3000 iot-api    # run container, map port 3000
```

Now your Flask app runs identically on any machine with Docker installed.

Docker Compose — Running Multiple Services

Our IoT system has many services:

Flask REST API	→ Container 1
MQTT Subscriber	→ Container 2
Mosquitto Broker	→ Container 3
PostgreSQL	→ Container 4
InfluxDB	→ Container 5
NGINX	→ Container 6

Docker Compose lets you define and start all containers with one command.

Starting the Entire System

```
# Start all services in background  
docker compose up -d
```

```
# Check running containers  
docker compose ps
```

```
# View logs from all services  
docker compose logs -f
```

```
# View logs from one service  
docker compose logs -f api
```

```
# Stop everything  
docker compose down
```

Service-to-Service Communication

Inside Docker Compose, containers talk to each other by **service name** — not IP address.

```
docker-compose.yml service names:
```

```
  mosquitto, api, mqtt-subscriber, postgres, influxdb, nginx
```

```
From inside "api" container:
```

```
  Database URL: postgresql://iotuser:secret@postgres:5432/iotdb
                                     ^^^^^^^^^
```

service name, not IP!

```
  MQTT broker:  mqtt.connect("mosquitto", 1883)
                             ^^^^^^^^^^^
```

service name!

Environment Variables for Configuration

Never hardcode passwords or hostnames. Use environment variables:

```
# api/main.py
import os

DATABASE_URL = os.environ.get("DATABASE_URL", "postgresql://localhost/iotdb")
MQTT_BROKER   = os.environ.get("MQTT_BROKER", "localhost")
SECRET_KEY    = os.environ.get("SECRET_KEY", "dev-only-key")

app = Flask(__name__)
```

We need to specify the environment variables in `docker-compose.yml`:

```
# docker-compose.yml
api:
  environment:
    - DATABASE_URL=postgresql://iotuser:secret@postgres:5432/iotdb
    - MQTT_BROKER=mosquitto
    - SECRET_KEY=production-secret-key-here
```

Same code works in development (localhost) and production (Docker) by changing env vars only.

Volumes — Persistent Data

Containers are **stateless** by default — data is lost when container stops.

Without volume:

Container starts → Writes data → Container stops → Data GONE

With volume:

Container starts → Writes to /var/lib/postgresql/data

mapped to host disk

Container stops → Data stays on disk ✓

Container restarts → Data still there ✓

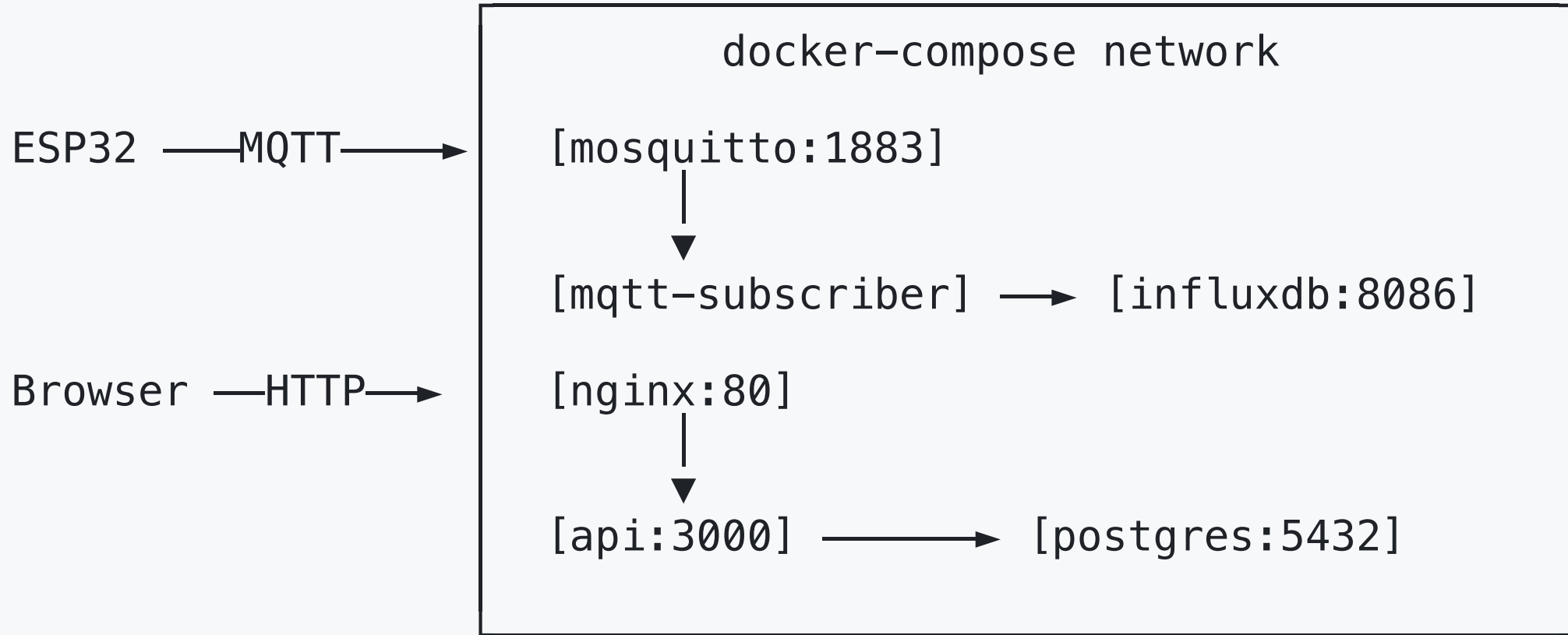
Using Volumes in `docker-compose.yml`, we can connect the PostgreSQL data (or any data) directory to a named volume:

```
postgres:
  volumes:
    - postgres_data:/var/lib/postgresql/data # named volume
    - ./init.sql:/docker-entrypoint-initdb.d/init.sql # init script
```

Project Directory Structure

```
iot-system/  
├── docker-compose.yml  
  
├── api/  
│   ├── Dockerfile  
│   ├── requirements.txt  
│   └── main.py  
  
├── mqtt-subscriber/  
│   ├── Dockerfile  
│   ├── requirements.txt  
│   └── mqtt_subscriber.py  
  
├── mosquitto/  
│   └── mosquitto.conf  
  
├── nginx/  
│   └── nginx.conf  
  
└── dashboard/  
    └── index.html
```

Complete IoT Architecture with Docker



Host machine only exposes: port 80 (nginx), port 1883 (mqtt)
Everything else stays internal to Docker network.

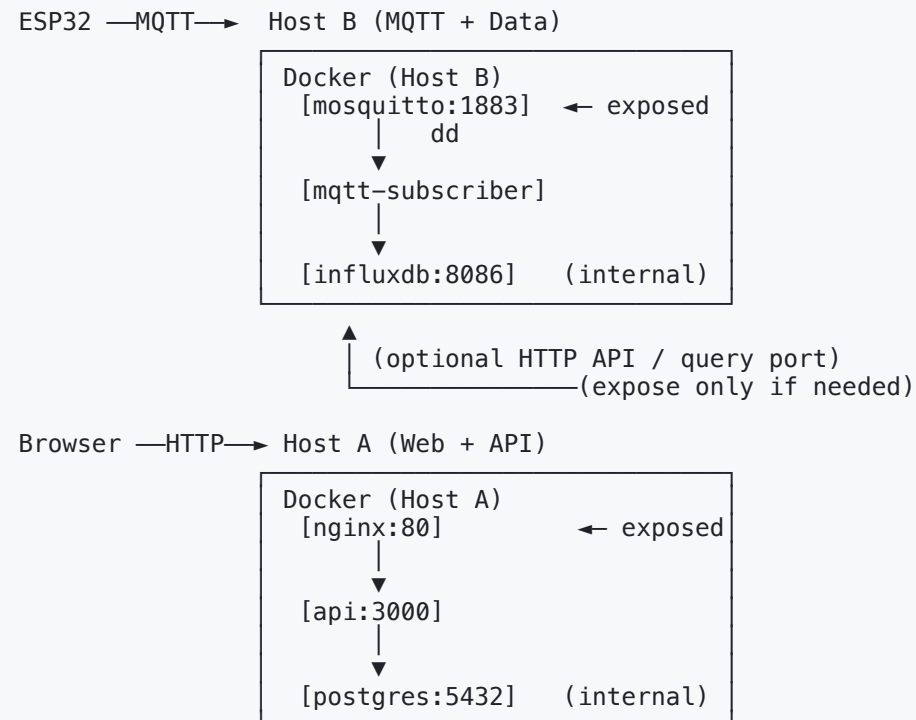
marp: true
theme: default

Multi-Host Docker Deployment (Not One Compose Network)

Goal: run each stack on **different machines** (or VMs), not one shared docker-compose network.

Architecture (Separated Hosts)

We can run the MQTT stack on one server (Host B) and the REST API + DB stack on another server (Host A):



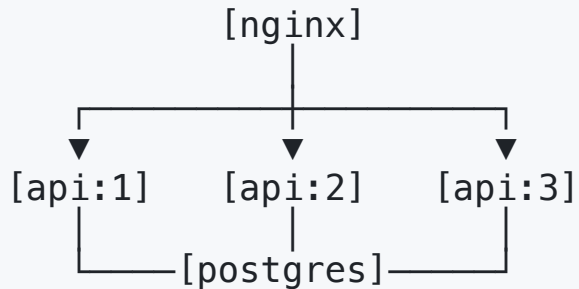
Between Host A and Host B:

- API can talk to MQTT broker using Host B IP/DNS (e.g., mqtt.example.net:1883)
- Or API can query InfluxDB via a secured exposed port (only if required)

Scaling Services

Need to handle more sensor traffic? Scale just the API:

```
# Run 3 instances of the API service
docker compose up -d --scale api=3
```



NGINX automatically load-balances across all API instances.